資料結構・課堂 5　Linked List 鏈結串列　測驗

一、選擇題　（每題 5 分，共 25 分）

1. Linked List 的每個節點包含哪兩個部分？

(　　) A. 鍵值（key）與雜湊碼（hash）

(　　) B. 資料（data）與指向下一個節點的指標（next）

(　　) C. 索引（index）與資料（data）

(　　) D. 前後兩個指標（prev 與 next）

---

2. 對 Singly Linked List 的**頭部**插入一個新節點，時間複雜度是？

(　　) A. O(n)　B. O(log n)　C. O(n^2)　D. O(1)

---

3. 下列哪個操作，Linked List 比 Array **更有效率**？

(　　) A. 用索引隨機存取 arr[i]

(　　) B. 在**已知位置**的節點後插入新節點

(　　) C. 搜尋某個特定值

(　　) D. 利用 CPU Cache 加速存取

---

4. Doubly Linked List 相較於 Singly Linked List 的主要優勢是？

(　　) A. 佔用更少記憶體

(　　) B. 可以雙向遍歷，且尾部刪除達到 O(1)

(　　) C. 搜尋速度更快

(　　) D. 不需要使用指標

---

5. 何時**最適合**選擇 Linked List 而非 Array？

(　　) A. 需要頻繁用索引讀取資料

(　　) B. 需要盡量減少記憶體使用

(　　) C. 資料大小無法預測，且頻繁在頭部或中間插入/刪除

(　　) D. 需要 Cache-Friendly 的存取模式

二、繪圖題　（共 24 分）

6.（12 分） 請畫出以下程式碼建立的 Singly Linked List 結構圖，標示 `head` 指向、各節點的 `data` 值、`next` 方向，以及最後的 `NULL`。

```
Node* n1 = new Node(); n1->data = 10;
Node* n2 = new Node(); n2->data = 30;
Node* n3 = new Node(); n3->data = 20;
n1->next = n2;
n2->next = n3;
n3->next = nullptr;
Node* head = n1;
```

（在下方空白處作圖）

7.（12 分） 在下圖 Linked List 中，在節點 `20` 後面插入新節點 `99`。請畫出插入後的結構，並標示哪些指標發生改變。

原始：

```
head → [10] → [20] → [30] → NULL
```

插入後（請畫出）：

指標變化說明：

① 節點 _ _ _ 的 next 從指向 _ _ _ 改為指向 _ _ _

② 新節點 [99] 的 next 指向 _ _ _

三、複雜度表格　（每格 3 分，共 27 分）

請填寫各操作的時間複雜度，並在「誰贏？」欄填入 Array / Linked List / 平手。

操作	Array	Singly LL	誰贏？
隨機存取 arr[i]	O(1)		
搜尋（未排序）	O(n)		
頭部插入	O(n)		
尾部插入（有 tail 指標）	O(1)		
中間插入（已知位置）	O(n)		
中間插入（需先搜尋）	O(n)		
頭部刪除	O(n)		
尾部刪除（Singly 無 prev）	O(1)		
Cache 效能	優秀		

四、簡答與分析題　（共 24 分）

8.（12 分） 說明 Singly、Doubly、Circular 三種 Linked List 的差異，各舉一個應用場景。

Singly Linked List（單向）：

特徵：每個節點有 _ _ _ 個指標（next）

應用場景：_ _

Doubly Linked List（雙向）：

特徵：每個節點有 _ _ _ 個指標（prev + next）

應用場景：_ _

Circular Linked List（環狀）：

特徵：最後一個節點的 next 指向 _ _ _ _ _ _ _ _ _ _

應用場景：_ _

9.（12 分） 給定 Linked List：`head → [5] → [3] → [8] → [1] → NULL`

請描述**刪除值為 3 的節點**的完整步驟，說明需要修改哪個指標，以及操作後的結構。

步驟①：

_ _

步驟②：

_ _

步驟③：

_ _

刪除後的結構：`head → [_ _] → [_ _] → [_ _] → NULL`

此操作的時間複雜度：O(_ _)　理由：_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _